Construcción de una Pokédex mediante una red neuronal convolucional (CNN)

Jorge Garzón
Instituto Tecnológico de Estudios Superiores de Monterrey
Inteligencia Artificial Avanzada para la Ciencia de Datos

05 de Noviembre de 2026

Resumen

La clasificación automática de imágenes permite identificar y distinguir categorías a partir de sus características visuales, aun con variaciones en su apariencia y representación. En este trabajo se aborda el reconocimiento de distintas especies Pokémon mediante aprendizaje profundo con TensorFlow, utilizando imágenes etiquetadas de Kaggle. El desarrollo comprende la preparación de los datos, la construcción de una red convolucional de referencia y la comparación de alternativas mediante transferencia de aprendizaje. La experimentación contempla aumento de datos, ajuste de hiperparámetros, regularización y fine-tuning para identificar mejoras en el clasificador. La evaluación utiliza conjuntos de entrenamiento, validación y prueba para analizar la capacidad de generalización y los errores entre clases. Finalmente, el proyecto incluye la exportación del modelo seleccionado y su uso para generar predicciones sobre imágenes proporcionadas por los usuarios.

1 Introducción

La identificación de categorías en imágenes constituye un problema de visión por computadora en el que deben reconocerse características como forma, color y distribución de sus elementos. En el caso de Pokémon, una misma clase puede aparecer como ilustración, modelo tridimensional o fotografía de una figura, con distintas poses y fondos. A su vez, algunas especies comparten rasgos que pueden dificultar su distinción.

El objetivo del proyecto es implementar un modelo de aprendizaje profundo con TensorFlow para asignar a cada imagen una de las 150 etiquetas disponibles. El problema se plantea como clasificación multiclase de una sola etiqueta por imagen. La clase que se desea predecir corresponde al nombre de la carpeta de origen, mientras que la entrada está formada por los píxeles que conforman la imagen. El alcance actual se limita a esas clases conocidas, por lo que no se plantea detectar objetos ni reconocer a cualquier Pokémon fuera del conjunto de datos.

Antes de definir y entrenar el modelo, es necesario examinar la calidad de las imágenes y la consistencia de sus etiquetas. La presencia de copias o versiones muy similares en particiones diferentes podría favorecer una evaluación demasiado optimista. Además, los tamaños y modos de color requieren un tratamiento uniforme. Por esta razón, el desarrollo parte del análisis exploratorio y la preparación de los datos, para después evaluar el desempeño inicial y las mejoras en el modelo.

2 Conjunto de Datos

Para abordar el problema de clasificación, se buscó una colección de imágenes que permitiera distinguir diferentes Pokémon a partir de sus características visuales. La variedad de poses, fondos y estilos de representación introduce diferencias entre imágenes de una misma clase, mientras que algunas clases comparten colores o formas similares. Estas condiciones permiten estudiar cómo un modelo de aprendizaje profundo aprende patrones que ayudan a diferenciar categorías, incluso cuando las imágenes presentan variaciones entre ellas.

Se seleccionó el dataset pokemonclassification, publicado en Kaggle, ya que cumple con las características deseadas. La descarga se realizó mediante kagglehub y se fijó la versión 1 para mantener la misma fuente durante el desarrollo. La carpeta PokemonData contiene un subdirectorio por clase y archivos de imagen en su interior. El inventario de esta versión cuenta con 6,837 archivos, todos reconocidos como imágenes.

El dataset incluye los Pokémon de la primera generación, excepto a Nidoran hembra y macho, y añade a Sandslash en su forma Alola como una clase independiente. Así, las 150 clases no corresponden exactamente a los 151 Pokémon originales, sino a la selección y organización del dataset. Las principales características del conjunto de datos se resumen en la Tabla 1:

Característica	Valor
Imágenes originales	6 837
Clases	150
Archivos originales	433.06 MiB
Mínimo por clase	26
Mediana por clase	46
Máximo por clase	66

Table 1: Características iniciales del conjunto de datos

En conjunto, el dataset reúne 6,837 imágenes distribuidas en 150 clases, con distintas cantidades de ejemplos por categoría.

3 Análisis Exploratorio de Datos

En esta etapa se realizó un análisis exploratorio para comprender la composición del dataset e identificar problemas antes de su procesamiento. Para ello, se construyó un inventario con la clase, el formato, las dimensiones, el modo de color y la transparencia de cada imagen. Esta información permitió realizar el análisis de imágenes sin necesidad de modificar las originales.

3.1 Distribución de Clases

La cantidad de imágenes por clase varía entre 26 y 66, con una razón de 2.54 entre la mayor y la menor representación. Las clases más representadas son Venusaur con con 66 imágenes y Pikachu con 62, mientras que las clases menos representadas son son Alakazam, Dratini y Meowth que tienen tan solo 26 cada uno. La mediana del dataset es de 46 imágenes por clase.

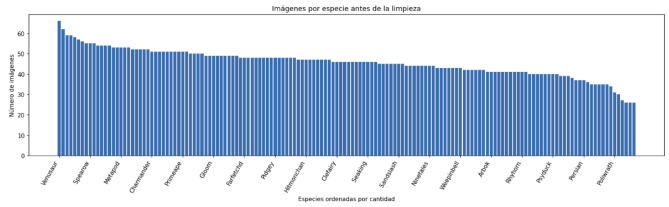


Figure 1: Imágenes por clase antes de la limpieza

Aunque ninguna clase concentra la mayoría de las observaciones, su representación no es uniforme y el número de ejemplos por categoría es limitado. Esta condición justifica conservar la distribución de etiquetas en las particiones y considerar métricas que permitan examinar el desempeño entre clases, además de la exactitud global. Asimismo, la cantidad limitada de ejemplos por clase motiva evaluar técnicas de aumento de datos en el entrenamiento, con el objetivo de introducir variaciones que pudieran favorecer a la generalización del modelo.

3.2 Dimensiones y Formatos

Las dimensiones de las imágenes del dataset presentan una variación considerable. La mediana del ancho es de 355 píxeles y la del alto es de 354, pero existen archivos con lados de un poco más de 40 píxeles y otros que alcanzan arriba de 4000.

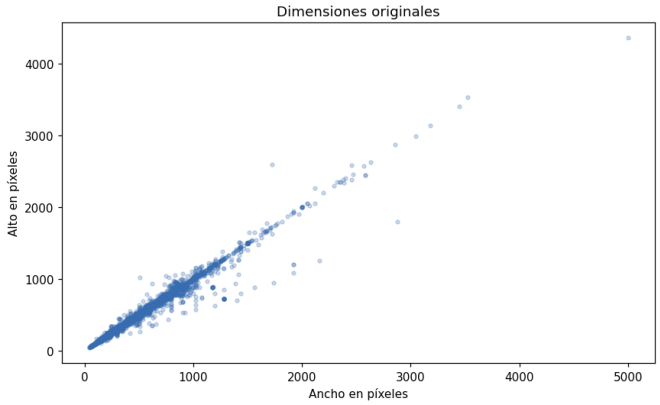


Figure 2: Dimensiones originales de las imágenes

La Tabla 2 muestra los extremos observados para cada dimensión por separado (ancho y alto). Los valores mínimos y máximos identificados no son necesariamente de la misma imagen.

Dimensión	Mínimo	Mediana	Máximo
Ancho	43	355	5,000
Alto	42	354	4,356

Table 2: Dimensiones originales en píxeles

También se identificaron 16 imágenes con al menos un lado menor de 64 píxeles y relaciones de aspecto entre 0.504 y 2.000. Estos resultados motivaron conservar la proporción al redimensionar, en lugar de estirar todas las imágenes a una forma cuadrada. Además, la baja resolución de una imagen no se utilizó por sí sola como criterio de exclusión, aunque ampliar estas imágenes no hace que recuperen el detalle perdido.

Finalmente, se identificaron 6710 imágenes JPEG, 115 PNG y 12 SVG. Entre las JPEG, 6707 se encuentran en modo RGB y 3 en escala de grises. Además, se detectaron 120 imágenes con transparencia y 124 con modo de color distinto de RGB. No se encontraron errores de encabezado ni archivos con más de un cuadro. Estos resultados exponen la necesidad de convertir las imágenes a un formato común y componer las transparencias observadas, para obtener entradas consistentes para el modelo.

Formato	Imágenes
JPEG	6710
PNG	115
SVG	12
Total	6837

Table 3: Formatos de las imágenes originales

3.3 Inspección Visual

La inspección visual incluyó una muestra de clases, imágenes con resolución reducida y casos con proporciones extremas. Como se observa en la Figura 3, la colección combina estilos y fondos diferentes. También pueden aparecer texto o elementos adicionales alrededor del personaje, lo que introduce ruido y señales que el modelo podría utilizar además de su apariencia.

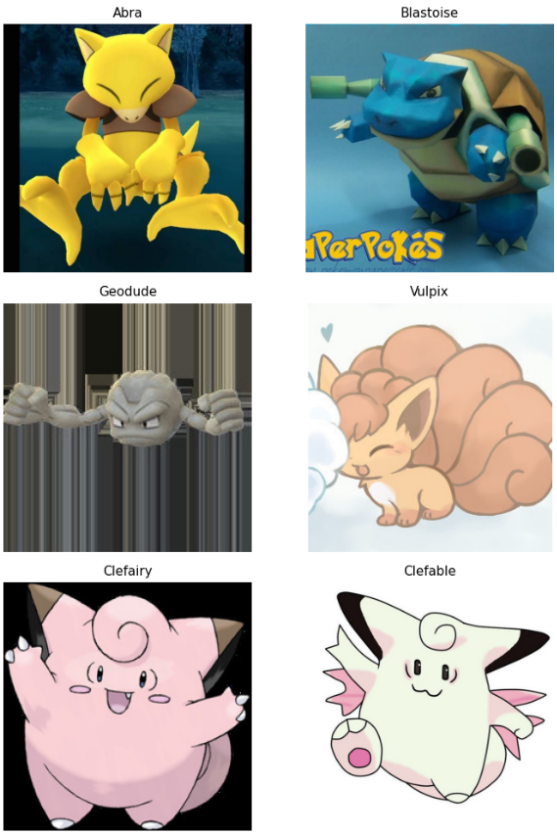


Figure 3: Muestras de imágenes

3.4 Duplicados y etiquetas

La presencia de imágenes repetidas puede aumentar artificialmente la representación de ciertos ejemplos y afectar la evaluación si sus copias aparecen en conjuntos diferentes. Por esta razón, se revisó la redundancia mediante huellas SHA-256 del archivo y de los píxeles, después de corregir la orientación y convertir a RGB. La igualdad de píxeles permite detectar copias aunque sus nombres o metadatos sean diferentes. Se identificaron 24 archivos pertenecientes a conjuntos de copias exactas. Esta cantidad considera tanto copias repetidas como imágenes a conservar, por lo que no todas deben de eliminarse.

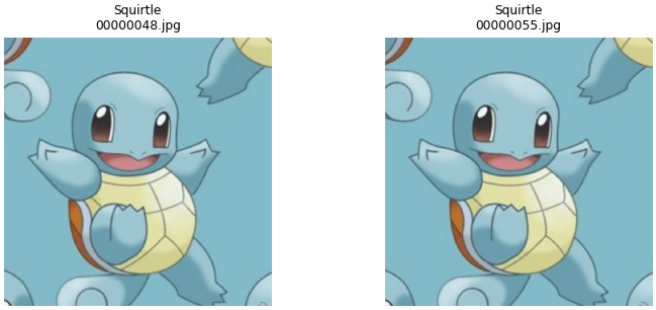


Figure 4: Ejemplo de imágenes duplicadas

Además, se implementaron dos huellas perceptuales de 64 bits: pHash, basado en frecuencias de la imagen, y dHash, basado en diferencias entre píxeles vecinos. Se propuso como candidato cada par con distancia de hasta 4 en pHash y hasta 6 en dHash, excluyendo los pares ya idénticos en píxeles. Este procedimiento produjo 31 pares para inspección visual. Esta similitud se utilizó como señal de revisión y agrupamiento, no como una prueba automática de duplicidad o error de etiqueta.

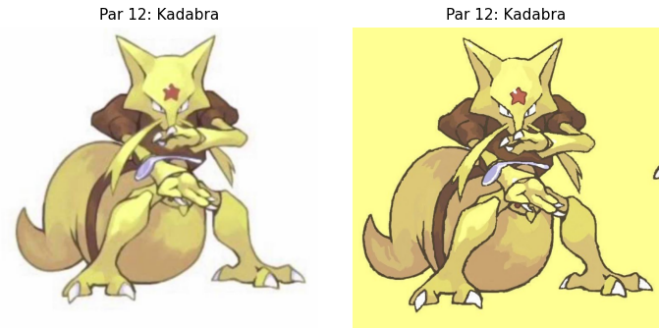


Figure 5: Ejemplo de imágenes similares

La inspección de los candidatos y de otros casos señalados dio lugar a exclusiones manuales por redundancia o por no corresponder a la clase indicada. La revisión manual fue dirigida a los casos identificados, por lo que no constituye una verificación exhaustiva de todas las imágenes del conjunto de datos.

3.5 Validación de Archivos

Finalmente, se realizó una validación para comprobar que las imágenes pudieran abrirse y procesarse correctamente. Esta revisión verificó el formato y decodificó los píxeles de cada archivo, permitiendo detectar posibles daños o problemas de lectura. Los 6,837 archivos superaron la validación, de modo que no fue necesario excluir imágenes por errores en la lectura.

4 Procesamiento de Datos

A partir de los problemas identificados durante el análisis exploratorio de datos se realizó la limpieza, se definieron particiones fijas y se generaron copias con una representación uniforme. Las decisiones se aplicaron antes de pasar al entrenamiento y los archivos de origen se conservaron sin cambios.

4.1 Limpieza del Conjunto

La limpieza atendió la redundancia y los problemas de etiquetado detectados anteriormente. Como parte de la misma, se excluyeron manualmente 35 imágenes: 9 eran duplicados presentes en clases distintas, de los cuales se retiró la copia con etiqueta incorrecta, y otras 26 se descartaron por redundancia visual. Después se eliminaron 7 copias exactas dentro de la misma clase, conservando solo un representante por conjunto.

Etapa	Excluidas	Restantes
Conjunto original	0	6837
Duplicados con etiqueta incorrecta	9	6828
Redundancia visual	26	6802
Copias exactas adicionales	7	6795

Table 4: Reducción del conjunto durante la limpieza

En total se retiraron 42 imágenes y se conservaron 6795, manteniendo las 150 clases. Como se mencionó antes, las señales de baja resolución o formato diferente no se utilizaron como motivos de exclusión por sí solas. Tampoco se realizó balanceo por sobremuestreo o submuestreo. Los nombres de clase se ordenaron y se asociaron a índices enteros, conservando esa relación para las futuras etapas de entrenamiento y predicción.

4.2 Partición de Conjuntos

Para entrenar y evaluar el modelo, los datos se dividieron en conjuntos de entrenamiento, validación y prueba. Esta separación permite evaluar su desempeño con imágenes que el modelo no utilizó para aprender. Además, se buscó evitar que copias o versiones muy similares aparecieran en conjuntos distintos, lo que podría producir resultados demasiado optimistas.

Para ello, se agruparon las imágenes que compartían los mismos píxeles, se volvían idénticas después del preprocesamiento o presentaban una alta similitud. Las imágenes de cada grupo se mantuvieron en una sola partición. Esta medida reduce el riesgo de compartir imágenes relacionadas entre distintos conjuntos, aunque depende de las similitudes detectadas.

La división se realizó buscando mantener proporciones similares de cada clase sin separar los grupos. Para ello, se generaron siete pliegues: uno para validación, otro para prueba y cinco que se unieron en un único conjunto de entrenamiento. Se probaron 20 configuraciones aleatorias y se eligió la más cercana al objetivo de 70/15/15 por cada clase, asegurando que cada una de las clases estuviera presente en los tres conjuntos.

La selección se basó únicamente en la distribución de las imágenes, sin utilizar resultados de modelos. Las proporciones finales de los conjuntos fueron aproximadamente de 71.4% para entrenamiento, 14.3% para validación y de 14.3% para prueba.

Conjunto	Imágenes	Porcentaje	Clases
Entrenamiento	4853	71.42%	150
Validación	972	14.30%	150
Prueba	970	14.28%	150
Total	6795	100.00%	150

Table 5: Distribución final de las particiones

Se comprobó que las rutas fueran únicas, que ningún grupo apareciera en más de una partición y que las 150 clases estuvieran presentes en las tres. Entrenamiento se utilizará para ajustar los pesos, validación para comparar configuraciones y prueba para la evaluación final. Debido al número limitado de imágenes por clase, el desempeño individual debe interpretarse considerando cuántos ejemplos se utilizaron para evaluar.

4.3 Preprocesamiento de Imágenes

Antes de estar listas para ingresarse al modelo, cada imagen se abrió, se corrigió su orientación EXIF y se compuso cualquier transparencia sobre un fondo blanco antes de convertirla a RGB. Después, se ajustó la imagen a un tamaño de 224 × 224 píxeles conservando su proporción. Para ello, la imagen redimensionada fue centrada en un lienzo blanco de dicho tamaño.

De esta manera se obtuvo una entrada de dimensiones constantes sin recortar el personaje ni deformar la proporción original de la imagen. La Figura 6 muestra uno de los ejemplos de entrenamiento usados para revisar la transformación. El procedimiento se aplicó por igual a las tres particiones ya que no requiere estimar ningún parámetro desde validación o prueba.

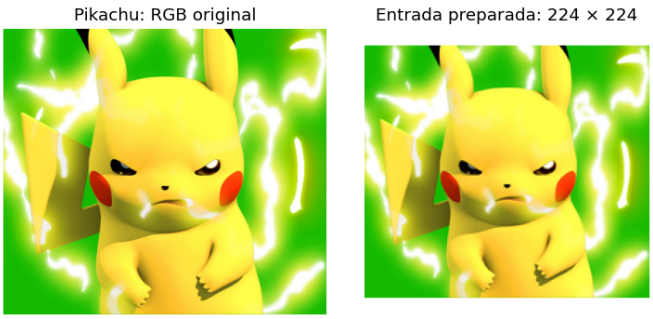


Figure 6: Ejemplo de una imagen preprocesada

Las copias finales de las imágenes se guardaron como PNG RGB, con tres canales y valores enteros en [0, 255]. En esta etapa no se aplicó normalización numérica ni aumento de datos. El escalado específico de cada arquitectura y la evaluación del uso de aumentación de datos se realizarán durante el modelado, dejando intactas las imágenes de validación/prueba.

4.4 Exportación

El conjunto preparado se exportó en un ZIP de 406.06 MiB con las 6,795 imágenes PNG, splits.csv y class_names.json. Estos archivos conservan las particiones, las etiquetas y la información necesaria para identificar y verificar las imágenes. Se comprobó la integridad del paquete, la correspondencia de la tabla exportada y la lectura de una imagen por partición como RGB de 224 × 224 en otra ubicación. Así, los datos quedaron listos para el entrenamiento y la evaluación de nuestro modelo.

5 Modelo Base

Como punto de partida se implementó una red neuronal convolucional (CNN) con TensorFlow y Keras. Esta arquitectura permite aprender patrones espaciales de las imágenes, como contornos, colores y combinaciones de formas, mediante filtros cuyos pesos se ajustan durante el entrenamiento.

Se considera un modelo base porque establece una referencia inicial para medir el efecto de las mejoras posteriores. Se eligió una arquitectura relativamente pequeña, entrenada desde cero y sin aumento de datos, Dropout, BatchNormalization ni pesos preentrenados. Esto permite documentar su comportamiento antes de incorporar otras técnicas, sin asumir que esta configuración sea la más adecuada para el problema.

5.1 Arquitectura del Modelo

La red recibe imágenes RGB de 224 × 224 píxeles. Su primera capa divide los valores entre 255 para así llevarlos al intervalo [0, 1]. Este escalado se integra en el modelo para que también se aplique al cargarlo y predecir la clase para imágenes nuevas.

La extracción de características utiliza tres bloques con 32, 64 y 128 filtros de 3 × 3, respectivamente. Cada convolución emplea ReLU para representar relaciones no lineales y se acompaña de MaxPooling de 2 × 2, que reduce a la mitad las dimensiones espaciales. El aumento progresivo de filtros permite disponer de más mapas de características mientras avanzamos.

Después se aplica GlobalAveragePooling2D, que resume cada mapa en un valor y evita conectar todas sus posiciones directamente al clasificador. Esta elección reduce la cantidad de parámetros frente a utilizar Flatten. Finalmente, una capa Dense de 128 unidades combina las características y una salida softmax genera una puntuación para cada una de las 150 clases, donde la predicción es la clase con mayor puntuación. En total, el modelo cuenta con 129110 parámetros entrenables.

Capa	Salida por imagen
Entrada RGB	$224 \times 224 \times 3$
Escalado $1/255$	$224 \times 224 \times 3$
Conv2D: 32 filtros, ReLU	$224 \times 224 \times 32$
MaxPooling2D: 2×2	$112 \times 112 \times 32$
Conv2D: 64 filtros, ReLU	$112 \times 112 \times 64$
MaxPooling2D: 2×2	$56 \times 56 \times 64$
Conv2D: 128 filtros, ReLU	$56 \times 56 \times 128$
MaxPooling2D: 2×2	$28 \times 28 \times 128$
GlobalAveragePooling2D	128 valores
Dense con ReLU	128 unidades
Dense con softmax	150 puntuaciones

Table 6: Arquitectura del modelo base

5.2 Configuración y Entrenamiento

Se conservaron las particiones previamente definidas: 4853 imágenes para entrenamiento, 972 para validación y 970 para prueba. Las imágenes se cargaron en lotes de 32, mezclando únicamente el conjunto de entrenamiento. La ejecución se realizó en un entorno de Google Colab haciendo uso de la GPU.

Como optimizador, se utilizó Adam con una tasa de aprendizaje inicial de 0.001 para actualizar los pesos. Como función de pérdida se seleccionó SparseCategoricalCrossentropy, adecuada para una clasificación multiclase. Esta función penaliza que el modelo asigne puntuaciones bajas a las clases correctas.

Se estableció un máximo de 30 épocas y se incorporó EarlyStopping para detener el entrenamiento después de cinco épocas consecutivas sin mejorar la pérdida de validación y se guardan los pesos asociados con su menor valor. De esta manera, el modelo evaluado corresponde a la mejor época según ese criterio, que no necesariamente coincide con la de mayor accuracy. Los valores iniciales son una configuración de referencia y no los resultados de una búsqueda de hiperparámetros.

Parámetro	Configuración
Optimizador	Adam
Tasa de aprendizaje	0.001
Tamaño del lote	32 imágenes
Máximo de épocas	30
Semilla	42
Pérdida	SparseCategoricalCrossentropy
Selección del checkpoint	Menor pérdida de validación
Paciencia de EarlyStopping	5 épocas

Table 7: Configuración inicial del entrenamiento

El entrenamiento terminó en la época 21, después de aproximadamente 5.30 minutos. Se seleccionó el checkpoint de la época 16 como modelo base. La validación intervino en esta selección, mientras que el conjunto de prueba se evaluó después de fijar los pesos del modelo. La Figura 7 muestra la evolución de la pérdida y la exactitud. La línea discontinua señala la época seleccionada por menor pérdida de validación.

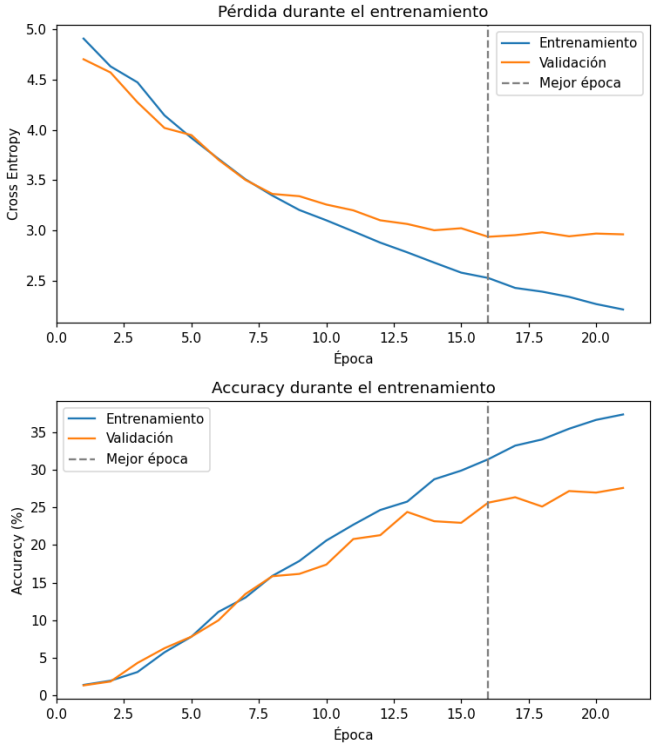


Figure 7: Accuracy y Pérdida durante el entrenamiento

5.3 Métricas de Evaluación

El desempeño se evaluó mediante accuracy, precision, recall, y F1, además de la pérdida. Accuracy expresa la proporción de imágenes clasificadas correctamente. Precision indica qué proporción de las predicciones asignadas a una clase es correcta, mientras que recall mide cuántas imágenes de esa clase se identifican. F1 combina ambas para examinar su equilibrio.

Precision, recall y F1 se calcularon por clase insividual y después se promediaron sin ponderación, obteniendo sus versiones macro. Así, cada Pokémon tiene el mismo peso en estas métricas, independientemente de su cantidad de imágenes. Este criterio complementa la exactitud global y permite observar un desempeño desigual entre categorías. F1 macro de validación se utilizará como criterio principal para comparar las futuras variantes y experimentos, manteniendo la pérdida de validación para seleccionar los checkpoints para cada modelo.

También se incluyó top-5 accuracy para medir cuántas veces la clase real aparece entre las cinco alternativas con mayor puntuación. Esta métrica permite evaluar una lista de candidatos útil para la Pokédex, pero no sustituye la evaluación de la predicción principal. Las métricas de entrenamiento y validación se compararon para observar la generalización, y el análisis para el planteamiento de mejoras se hizo en validación.

5.4 Resultados del Modelo Base

Los tres conjuntos se evaluaron con los mismos pesos de la mejor época seleccionada. Por ello, las cifras de la Tabla 8 corresponden a una evaluación posterior del checkpoint seleccionado y no a las métricas acumuladas mientras los pesos cambiaban durante el entrenamiento. La pérdida está expresada en su escala original y las demás métricas en porcentajes.

Métrica	Entrenamiento	Validación	Prueba
Pérdida	2.3703	2.9363	3.0626
Accuracy	35.19%	25.62%	22.78%
Precision macro	37.07%	24.35%	20.59%
Recall macro	34.07%	26.96%	23.42%
F1 macro	31.31%	22.69%	18.83%
Top-5 accuracy	70.64%	56.89%	55.15%

Table 8: Desempeño del modelo base

El modelo obtuvo un accuracy de 35.19% en entrenamiento, frente a 25.62% en validación y 22.78% en prueba. La diferencia de 9.58 puntos porcentuales entre entrenamiento y validación muestra una generalización limitada. Esta diferencia también se observa en F1 macro, que disminuye de 31.31% a 22.69% entre ambos conjuntos. Las métricas de validación servirán como referencia para comparar con los experimentos.

Como se observa en la Figura 7, la pérdida de entrenamiento continúa disminuyendo mientras la de validación presenta mejoras menores y oscilaciones en las últimas épocas. Junto con la brecha entre conjuntos, este comportamiento sugiere overfitting. Sin embargo, la exactitud de entrenamiento también permanece limitada, por lo que no puede atribuirse todo el problema exclusivamente al overfitting ni suponerse que entrenar durante más épocas bastaría para resolver esta situación.

En prueba, top-5 accuracy alcanzó 55.15%, frente a 22.78% para la predicción principal. Esto muestra que, en parte de los errores, la clase correcta sí aparece entre las primeras alternativas, aunque no reciba la mayor puntuación. El resultado aporta información sobre la clasificación, pero todavía muestra capacidad limitada para distinguir precisamente las 150 clases.

5.5 Análisis de Errores

La matriz de confusión de validación cruza las etiquetas reales con las predicciones de las 150 clases. La diagonal reúne los aciertos y las demás celdas indican las clases con las que se confundieron las imágenes. En total se registraron 249 aciertos y 723 errores entre las 972 imágenes de validación evaluadas.

Para facilitar su lectura, la Figura 8 muestra solamente las diez clases con menor F1, normalizando cada fila por su número de imágenes. La columna Otras agrupa las predicciones hacia las 140 clases que no aparecen individualmente. Todas las clases seleccionadas tienen recall y F1 iguales a cero; por ejemplo, Abra, Aerodactyl y Butterfree cuentan con 8, 5 y 2 imágenes, respectivamente. Estos resultados deben interpretarse considerando el reducido número de imágenes por clase.

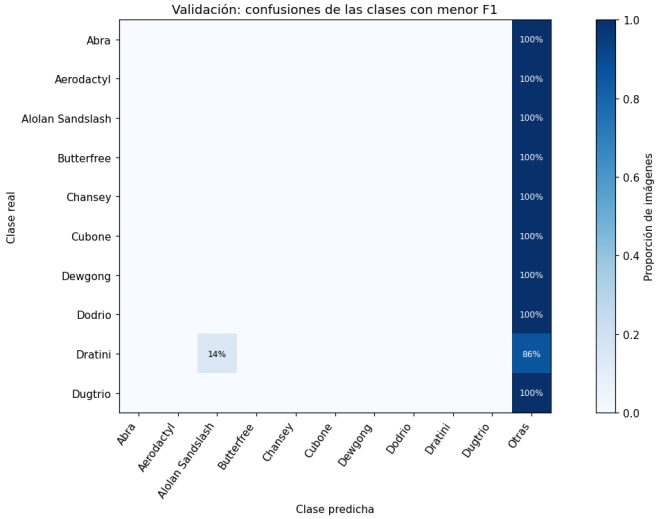


Figure 8: Matriz de confusión de validación

En nueve de las diez filas, el 100% de las predicciones se concentra en Otras. En el caso de Dratini, una de siete imágenes se asignó a Alolan Sandslash y las seis restantes a Otras. Por ello, esta matriz resumida nos permite identificar clases sin aciertos, pero no detalla todos sus destinos de error. Al consultar la matriz completa se observan cinco casos de Jigglypuff clasificados como Wigglytuff y cinco de Ponyta como Rapidash. Estos conteos describen confusiones concretas entre especies similares, sin probar del todo qué características las generaron.

La galería de la Figura 9 complementa a la matriz de confusión mostrando errores de puntuación softmax alta. Por ejemplo, una imagen de Magnemite se clasificó como Bulbasaur con una puntuación de 0.937. Esto nos indica que una puntuación alta no garantiza que la predicción sea correcta ni representa una confianza alta. La galería es una selección de algunos de los errores y no una muestra representativa de sus frecuencias.



Figure 9: Errores de predicción en validación

Estos resultados establecen una referencia inicial para evaluar si técnicas como el aumento de datos, la regularización y la transferencia de aprendizaje permiten mejorar la clasificación. Los errores observados y las diferencias de desempeño entre entrenamiento y validación orientarán la selección de los experimentos. Las variantes se compararán utilizando las mismas particiones y métricas, para analizar el efecto de cada cambio bajo condiciones consistentes. Las decisiones de ajuste se tomarán únicamente con el conjunto de validación, reservando el conjunto de prueba para evaluar el modelo final.

5.6 Exportación del modelo

El checkpoint de la mejor época seleccionada se guardó en formato .keras, conservando la arquitectura, los pesos y el escalado. Se acompañó con dos archivos: class_names.json, con el orden de las etiquetas, y training_results.json, con la configuración, el historial, las métricas y la matriz de confusión. Los tres archivos se empaquetaron en base_model.zip, de aproximadamente 0.47 MiB. Se comprobó la integridad del ZIP y que las predicciones de un mismo lote coincidieran al guardar y volver a cargar el modelo. Este paquete permite conservar la referencia y utilizarla posteriormente para hacer predicciones.

6 Experimentos

Para mejorar el modelo base se realizaron cinco experimentos, resumidos en la Tabla 9. Las configuraciones se compararon mediante F1 macro de validación, conservando las particiones y reservando el conjunto de prueba solo para evaluación final.

Experimento	Motivo
E1. Comparación de Arquitecturas	Seleccionar la mejor arquitectura preentrenada
E2. Data Augmentation	Evaluar si el aumento de datos mejora la generalización
E3. Fine-Tuning del último bloque	Adaptar las características aprendidas a los Pokémon
E4. Tasas de Dropout	Evaluar el efecto de la regularización
E5. Ajuste de Hiperparámetros	Ajustar el lote y la magnitud de las actualizaciones

Table 9: Resumen de los experimentos de mejora

6.1 E1. Comparación de Arquitecturas

Debido al número limitado de imágenes que se tiene por clase, se evaluó la transferencia de aprendizaje para aprovechar características visuales aprendidas previamente en ImageNet. El propósito fue comparar arquitecturas preentrenadas y seleccionar una como referencia para realizar futuros experimentos.

Se compararon MobileNetV2, ResNet50 y EfficientNetB0. En cada arquitectura se retiró el clasificador original y se añadió GlobalAveragePooling2D seguido de una capa Dense con softmax para las 150 clases. Las bases permanecieron congeladas y solo se entrenó la capa de salida, sin agregar aumento de datos ni Dropout adicional. Cada modelo incorporó el preprocesamiento correspondiente para sus pesos preentrenados.

Modelo	Accuracy (%)	F1 macro (%)	Pérdida
CNN base	25.62	22.69	2.9363
MobileNetV2	77.26	76.57	0.9065
ResNet50	**84.16**	**83.34**	**0.6759**
EfficientNetB0	82.10	80.61	0.7289

Table 10: Comparación de arquitecturas en validación

ResNet50 obtuvo el mayor F1 macro, superando a EfficientNetB0 por 2.72 puntos porcentuales y a la CNN base por 60.65. Se seleccionó su checkpoint de la época 10, tras ejecutar 13 épocas. Aunque requirió aproximadamente 1.5 minutos más que EfficientNetB0, se priorizó el desempeño de clasificación. La comparación con la CNN base compara configuraciones completas, no aísla el efecto de los pesos preentrenados.

Finalmente, se conservó ResNet50 como referencia para las siguientes modificaciones. Esta decisión se limita al experimento probado: se utilizó una sola semilla y tanto MobileNetV2 como EfficientNetB0 alcanzaron la época 15 con su menor pérdida de validación. Por ello, los resultados no indican cuál sería mejor después de probar más configuraciones.

6.2 E2. Data Augmentation

Para intentar mejorar la generalización y reducir la diferencia entre el desempeño de entrenamiento y validación se incorporó data augmentation mediante volteos horizontales aleatorios, rotaciones de hasta ±10◦ y traslaciones de hasta ±5% en ambos ejes, con interpolación bilineal y relleno blanco. Las transformaciones se aplicaron únicamente a entrenamiento, antes del preprocesamiento de ResNet50. Se mantuvieron fijas las imágenes de evaluación y no se generaron más copias aumentadas.

Se repitió el entrenamiento con la base de ImageNet congelada, entrenando un clasificador nuevo, y conservando la semilla y los hiperparámetros de la referencia. El proceso terminó después de 13 épocas y seleccionó la época 10 como la mejor. Las métricas de la Tabla 11 corresponden a la evaluación de ambos checkpoints a partir de las mismas imágenes.

Métrica	Sin augmentation	Con augmentation
Accuracy (%)	84.16	82.61
Precision macro (%)	85.14	83.51
Recall macro (%)	84.45	82.44
F1 macro (%)	83.34	81.00
Top-5 accuracy (%)	94.34	94.44
Pérdida	0.6759	0.7139

Table 11: Efecto del aumento de datos en validación

El F1 macro disminuyó 2.34 puntos porcentuales y el accuracy cayó 1.54 puntos en comparación al experimento previo. Aunque top-5 accuracy aumentó un poco, la pérdida fue mayor y las métricas principales empeoraron respecto a la base. La Figura 10 nos muestra la comparación de pérdida entre ambas.

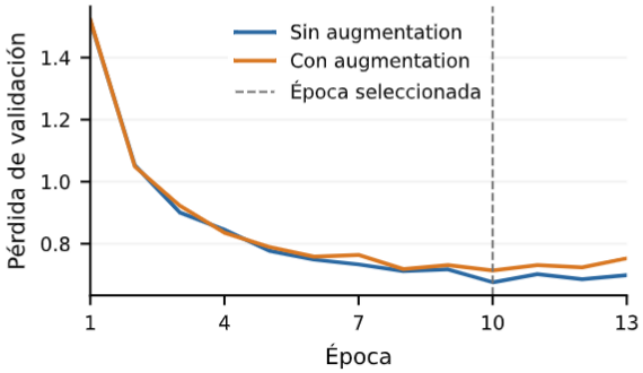


Figure 10: Pérdida de validación con y sin augmentation

Se decidió continuar con ResNet50 sin aplicar data augmentation y conservar el experimento solo como evidencia. Su incorporación redujo el F1 macro y el accuracy de validación, además de aumentar la pérdida respecto a la referencia. Aunque el top-5 accuracy presentó una mejora, esta no compensó que el desempeño de predicción principal haya bajado.

Por lo tanto, la aumentación probada se descartó de los siguientes experimentos, al no aportar una mejora en el desempeño de clasificación. Esta conclusión se limita a las transformaciones y condiciones evaluadas, otras transformaciones o su aplicación durante fine-tuning podrían dar resultados distintos.

6.3 E3. Fine-Tuning del Último Bloque

Para adaptar las características preentrenadas al conjunto de imágenes de Pokémon se retomó el checkpoint del primer experimento, sin augmentation. Esa etapa ya había entrenado el clasificador con la base congelada. En la segunda fase se habilitaron las diez capas convolucionales de la última etapa de ResNet50 (conv5_) y se continuó ajustando la capa de salida. Las etapas y las capas previas a éstas se quedaron congeladas.

Se recompiló el modelo con Adam y una tasa de aprendizaje de 10^{-5}, para realizar ajustes pequeños sobre los pesos existentes. Se permitieron hasta 15 épocas adicionales, con EarlyStopping de paciencia 4. Se ejecutaron siete épocas en 4.29 minutos y se seleccionó la tercera por su menor pérdida de validación. Las métricas presentadas en la Tabla 12 corresponden a dicha fase.

Métrica	Base congelada	Fine-tuning
Accuracy (%)	84.16	84.67
Precision macro (%)	85.14	85.51
Recall macro (%)	84.45	85.02
F1 macro (%)	83.34	83.97
Top-5 accuracy (%)	94.34	95.06
Pérdida	0.6759	0.6516

Table 12: Efecto del fine-tuning en validación

El F1 macro aumentó 0.63 puntos porcentuales y el accuracy 0.51 puntos, con 823 aciertos frente a 818 sobre las mismas 972 imágenes. También disminuyó la pérdida, aunque volvió a aumentar después de la época seleccionada. Estos resultados muestran una mejora leve con el ajuste de la última etapa, sin eliminar la diferencia entre entrenamiento y validación: el accuracy de entrenamiento llegó hasta 99.96% frente al 84.67%.

Finalmente, se conservó ResNet50 con fine-tuning como la mejor configuración obtenida hasta este experimento, por su mayor F1 macro y la mejora de las demás métricas de validación. Este checkpoint se utilizará como referencia para los siguientes ajustes y para la selección del modelo final. La mejora es pequeña y corresponde a una sola ejecución, el desempeño en prueba se evaluará al finalizar estos experimentos.